

Rahul Sunda (24b2147)

Problem List

53. Maximum Subarray

Solved

N/A

Topics

Companies

Given an integer array `nums`, find the **subarray** with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Code

Accepted

C++

Auto

```
1 class Solution {
2 public:
3     int maxSubArray(vector<int>& nums) {
4         int maxSum = INT_MIN;
5         int currentSum = 0;
6
7         for (int i = 0; i < nums.size(); i++) {
8             currentSum += nums[i];
9
10            if (currentSum > maxSum) {
11                maxSum = currentSum;
12            }
13
14            if (currentSum < 0) {
15                currentSum = 0;
16            }
17        }
18
19        return maxSum;
20    }
21 };
```

Saved

Ln 1, Col 1

Problem List

121. Best Time to Buy and Sell Stock

Solved

N/A

Topics

Companies

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: `prices = [7,6,4,3,1]`
Output: 0
Explanation: Since the prices are decreasing, no transaction can be made to achieve a profit.

Code

Accepted

C++

Auto

```
1 class Solution {
2 public:
3     int maxProfit(vector<int>& prices) {
4         int minprice=prices[0];
5         int maxprofit=0;
6         for (int i=0;i<prices.size();i++){
7             maxprofit=max(maxprofit,prices[i]-minprice);
8             minprice=min(prices[i],minprice);
9         }
10        return maxprofit;
11    }
12 }
13 };
```

Saved

Ln 1, Col 1

Problem List

Description

Editorial

Solutions

Submissions

73. Set Matrix Zeroes

Solved

N/A Topics Companies Hint

Given an $m \times n$ integer matrix `matrix`, if an element is `0`, set its entire row and column to `0`'s.

You must do it [in place](#).

Example 1:

1	1	1		1	0	1
1	0	1	→	0	0	0
1	1	1		1	0	1

Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`
Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0		0	0	0	0
3	4	5	2	→	0	4	5	0
1	3	1	5		0	3	1	0

Code

Accepted

C++

Auto

```
1 class Solution {
2 public:
3     void setZeroes(vector<vector<int>>& matrix) {
4         int m=matrix.size();
5         int n=matrix[0].size();
6         vector<bool> row(m, false);
7         vector<bool> col(n, false);
8
9         for(int i = 0; i < m; i++){
10             for(int j = 0; j < n; j++){
11                 if(matrix[i][j] == 0){
12                     row[i] = true;
13                     col[j] = true;
14                 }
15             }
16         }
17
18         for(int i = 0; i < m; i++){
19             for(int j = 0; j < n; j++){
20                 if(row[i] || col[j]){
21                     matrix[i][j] = 0;
22                 }
23             }
24         }
25     }
26 };
```

Saved

Ln 1, Col 1

Problem List

238. Product of Array Except Self

Solved

N/A

Topics

Companies

Hint

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`
Output: `[0,0,9,0,0]`

Constraints:

C++

Accepted

Auto

```
1 class Solution {
2 public:
3     vector<int> productExceptSelf(vector<int>& nums) {
4         int n = nums.size();
5         vector<int> res(n);
6
7         int pre = 1;
8         for(int i = 0; i < n; i++){
9             res[i] = pre;
10            pre *= nums[i];
11        }
12
13        int suf = 1;
14        for(int i = n - 1; i >= 0; i--){
15            res[i] *= suf;
16            suf *= nums[i];
17        }
18
19        return res;
20    }
21 };
22
23
```

Saved

Ln 1, Col 1

Problem List

1979. Find Greatest Common Divisor of Array

Solved

1184

Topics

Companies

Hint

Given an integer array `nums`, return the **greatest common divisor** of the smallest number and largest number in `nums`.

The **greatest common divisor** of two numbers is the largest positive integer that evenly divides both numbers.

Example 1:

Input: `nums = [2,5,6,9,10]`
Output: 2
Explanation:
The smallest number in `nums` is 2.
The largest number in `nums` is 10.
The greatest common divisor of 2 and 10 is 2.

Example 2:

Input: `nums = [7,5,6,8,3]`
Output: 1
Explanation:

C++

Accepted

Auto

```
1 class Solution {
2 public:
3     int findGCD(vector<int>& nums) {
4         int n=nums.size();
5         for(int i=0;i<n;i++){
6             for(int j=0;j<n-i-1;j++){
7                 if(nums[j+1]< nums[j]){
8                     swap(nums[j],nums[j+1]);
9                 }
10            }
11        }
12        int Max=nums[n-1];
13        int Min=nums[0];
14        while(Min!=0 && Max!=0){
15            if(Max>Min){Max=Max%Min;}
16            else if(Min>Max){Min=Min%Max;}
17            else if(Max==Min){return Max;}
18        }
19        if(Min==0){return Max;}
20        return Min;
21    }
22 };
23
24
```

Saved

Ln 1, Col 1

Problem List < > **Submit** **Premium**

Description | **Editorial** | **Solutions** | **Submissions**

You are given an $n \times n$ 2D `matrix` representing an image, rotate the image by **90 degrees** (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D `matrix` directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:

1	2	3
4	5	6
7	8	9

→

7	4	1
8	5	2
9	6	3

Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output: `[[7,4,1],[8,5,2],[9,6,3]]`

Example 2:

5	1	9	11		15	13	2	5
---	---	---	----	--	----	----	---	---

```

1  class Solution {
2  public:
3      void rotate(vector<vector<int>>& matrix) {
4          int n=matrix.size();
5          for(int i=0;i<n;i++){
6              for(int j=0;j<=i;j++){
7                  swap(matrix[i][j],matrix[j][i]);
8              }
9          }
10         for(int i=0;i<n;i++){
11             reverse(matrix[i].begin(),matrix[i].end());
12         }
13     }
14 };

```

Saved Ln 1, Col 1

Problem List < > **Submit** **Premium**

Description | **Editorial** | **Solutions** | **Submissions**

54. Spiral Matrix Solved

N/A | **Topics** | **Companies** | **Hint**

Given an $m \times n$ `matrix`, return all elements of the `matrix` in spiral order.

Example 1:

1	→	2	→	3
4	→	5		↓
↑				↓
7	←	8	←	9

Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output: `[1,2,3,6,9,8,7,4,5]`

Example 2:

1	→	2	→	3	→	4
---	---	---	---	---	---	---

```

1  class Solution {
2  public:
3      vector<int> spiralOrder(vector<vector<int>>& matrix) {
4          int n=matrix.size();
5          int m=matrix[0].size();
6          int left=0,right=m-1;
7          int top=0,bottom=n-1;
8          vector<int> v;
9          while(left<=right && top<=bottom){
10             for(int i=left;i<=right;i++){
11                 v.push_back(matrix[top][i]);
12             }
13             top++;
14             for (int i=top;i<=bottom;i++){
15                 v.push_back(matrix[i][right]);
16             }
17             right--;
18             if(top<=bottom){
19                 for(int i=right;i>=left;i--){
20                     v.push_back(matrix[bottom][i]);
21                 }
22                 bottom--;
23             }
24             if(left<=right){
25                 for(int i=bottom;i>=top;i--){
26                     v.push_back(matrix[i][left]);
27                 }
28                 left++;
29             }
30         }
31         return v;
32     }
33 };

```

Saved Ln 1, Col 1

DescriptionEditorialSolutionsSubmissions

1. Two SumSolved

N/ATopicsCompaniesHint

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have **exactly one solution**, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`
Output: `[0,1]`
Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`
Output: `[1,2]`

CodeAccepted

C++Auto

```
1 class Solution {
2 public:
3     vector<int> twoSum(vector<int>& nums, int target) {
4
5         for (int i=0;i<nums.size();i++){
6             for(int j=i+1;j<nums.size();j++){
7                 if (nums[j]== target - nums[i]){
8                     return {i,j};
9                 }
10            }
11        }
12        return {};
13    }
14
15
16 };
```

Problem List

DescriptionEditorialSolutionsSubmissions

509. Fibonacci NumberSolved

N/ATopicsCompanies

The **Fibonacci numbers**, commonly denoted $F(n)$ form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

$$F(0) = 0, F(1) = 1$$
$$F(n) = F(n - 1) + F(n - 2), \text{ for } n > 1.$$

Given `n`, calculate $F(n)$.

Example 1:

Input: `n = 2`
Output: `1`
Explanation: $F(2) = F(1) + F(0) = 1 + 0 = 1$.

Example 2:

Input: `n = 3`
Output: `2`
Explanation: $F(3) = F(2) + F(1) = 1 + 1 = 2$.

CodeAccepted

C++Auto

```
1 class Solution {
2 public:
3     int fib(int n) {
4         if (n==0){
5             return 0;
6         }
7         if (n==1){
8             return 1;
9         }
10        return fib(n-1)+fib(n-2);
11    }
12 };
```

Problem List

DescriptionEditorialSolutionsSubmissions

75. Sort ColorsSolved

N/ATopicsCompaniesHint

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: `nums = [2,0,2,1,1,0]`
Output: `[0,0,1,1,2,2]`

Example 2:

Input: `nums = [2,0,1]`
Output: `[0,1,2]`

CodeAccepted

C++Auto

```
1 class Solution {
2 public:
3     void sortColors(vector<int>& nums) {
4         int low=0,mid=0,high=nums.size()-1;
5         while(mid<=high){
6             if(nums[mid]==0){
7                 swap(nums[low],nums[mid]);
8                 low++;
9                 mid++;
10            } else if (nums[mid]==1){
11                mid++;
12            } else {
13                swap(nums[mid],nums[high]);
14                high--;
15            }
16        }
17    }
18 };
```

169. Majority Element

Solved

N/A Topics Companies

Given an array `nums` of size `n`, return the majority element.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`
Output: `3`

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`
Output: `2`

```
1 class Solution {
2 public:
3     int majorityElement(vector<int>& nums) {
4         int candidate, count = 0, n = nums.size();
5         for(int i = 0; i < n; i++) {
6             if(count == 0) candidate = nums[i];
7             if(nums[i] == candidate) count++;
8             else count--;
9         }
10        return candidate;
11    }
12 }
13
14
15 };
```

2149. Rearrange Array Elements by Sign

Solved

1235 Topics Companies Hint

You are given a **0-indexed** integer array `nums` of **even** length consisting of an **equal** number of positive and negative integers.

You should return the array of `nums` such that the array follows the given conditions:

- Every **consecutive pair** of integers have **opposite signs**.
- For all integers with the same sign, the **order** in which they were present in `nums` is **preserved**.
- The rearranged array begins with a positive integer.

Return the modified array after rearranging the elements to satisfy the aforementioned conditions.

Example 1:

Input: `nums = [3,1,-2,-5,2,-4]`
Output: `[3,-2,1,-5,2,-4]`
Explanation:
The positive integers in `nums` are `[3,1,2]`. The negative integers

```
1 class Solution {
2 public:
3     vector<int> rearrangeArray(vector<int>& nums) {
4         vector<int> arr(nums.size(),0);
5         int posindex=0;
6         int negindex=1;
7         for(int i=0;i<nums.size();i++){
8             if (nums[i]<0){
9                 arr[negindex]=nums[i];
10                negindex+=2;
11            } else {
12                arr[posindex]=nums[i];
13                posindex+=2;
14            }
15        }
16        return arr;
17    }
18 };
```

1752. Check if Array Is Sorted and Rotated

Solved

1324 Topics Companies Hint

Given an array `nums`, return `true` if the array was originally sorted in non-decreasing order, then rotated **some** number of positions (including zero). Otherwise, return `false`.

There may be **duplicates** in the original array.

Note: An array `A` rotated by `x` positions results in an array `B` of the same length such that `B[i] == A[(i+x) % A.length]` for every valid index `i`.

Example 1:

Input: `nums = [3,4,5,1,2]`
Output: `true`
Explanation: `[1,2,3,4,5]` is the original sorted array. You can rotate the array by `x = 2` positions to begin on the element of value 3: `[3,4,5,1,2]`.

Example 2:

Input: `nums = [2,1,3,4]`
Output: `false`

```
1 class Solution {
2 public:
3     bool check(vector<int>& nums) {
4         int count = 0;
5         int n = nums.size();
6
7         for (int i = 0; i < n; i++) {
8             if (nums[i] > nums[(i + 1) % n]) {
9                 count++;
10            }
11        }
12
13        return count <= 1;
14    }
15 };
```

Testcase Test Result

You must run your code first

Problem List

26. Remove Duplicates from Sorted Array

Solved

N/A

Topics

Companies

Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
```

19.3K 1.1K 375 Online

Code

Accepted

C++

Auto

```
1 class Solution {
2 public:
3     int removeDuplicates(vector<int>& nums) {
4         if (nums.empty()) {
5             return 0;
6         }
7
8         int k = 1;
9         for (int i = 1; i < nums.size(); i++) {
10             if (nums[i] != nums[i - 1]) {
11                 nums[k] = nums[i];
12                 k++;
13             }
14         }
15
16         return k;
17     }
18 }
```

Saving...

Ln 1, Col 1

Testcase

Test Result

You must run your code first

Problem List

189. Rotate Array

Solved

N/A

Topics

Companies

Hint

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

Input: `nums = [1,2,3,4,5,6,7], k = 3`
Output: `[5,6,7,1,2,3,4]`
Explanation:
rotate 1 steps to the right: `[7,1,2,3,4,5,6]`
rotate 2 steps to the right: `[6,7,1,2,3,4,5]`
rotate 3 steps to the right: `[5,6,7,1,2,3,4]`

Example 2:

Input: `nums = [-1,-100,3,99], k = 2`
Output: `[3,99,-1,-100]`
Explanation:
rotate 1 steps to the right: `[99,-1,-100,3]`
rotate 2 steps to the right: `[3,99,-1,-100]`

Code

Accepted

C++

Auto

```
1 class Solution {
2 public:
3     void rotate(vector<int>& nums, int k) {
4         int n=nums.size();
5         k=k%n;
6         vector<int> temp(k,0);
7         for(int l=0;l<k;l++){
8             temp[l]=nums[n-k+l];
9         }
10        for(int i = n - k - 1; i >= 0; i--){
11            nums[i + k] = nums[i];
12        }
13        for(int j=0;j<k;j++){
14            nums[j]=temp[j];
15        }
16    }
17    };
18 }
```

Saved

Ln 1, Col 1

Testcase

Test Result

You must run your code first

Problem List

283. Move Zeroes

Solved

Description

Editorial

Solutions

Submissions

N/A

Topics

Companies

Hint

Given an integer array `nums`, move all `0`'s to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`
Output: `[1,3,12,0,0]`

Example 2:

Input: `nums = [0]`
Output: `[0]`

Code

Accepted

C++

Auto

```
1 class Solution {
2 public:
3     void moveZeroes(vector<int>& nums) {
4         for(int i=0;i<nums.size();i++){
5             if (nums[i]==0){
6                 int j=i+1;
7                 while(j<nums.size() && nums[j]==0){
8                     j++;
9                 }
10                if (j < nums.size()) {
11                    swap(nums[i], nums[j]);
12                }
13            }
14        }
15    }
16 }
17 ;;
```

Saved

Ln 1, Col 1

Testcase

Test Result

Problem List

136. Single Number

Solved

Description

Editorial

Solutions

Submissions

N/A

Topics

Companies

Hint

Given a **non-empty** array of integers `nums`, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`
Output: `1`

Example 2:

Input: `nums = [4,1,2,1,2]`
Output: `4`

Example 3:

Input: `nums = [1,1]`
Output: `1`

Code

Accepted

C++

Auto

```
1 class Solution {
2 public:
3     int singleNumber(vector<int>& nums) {
4         int j=0;
5         int res=0;
6         while(j<nums.size()) {
7             res=res^nums[j];
8             j++;
9         }
10        return res;
11    }
12 }
13 ;;
```

Saved

Ln 1, Col 1

Testcase

Test Result

You must run your code first

18.9K

324

100 Online

Problem List

leetcode.com – to exit full screen, press Esc

Esc

Description

Editorial

Solutions

Submissions

9. Palindrome Number

Solved

N/A Topics Companies Hint

Given an integer `x`, return `true` if `x` is a **palindrome**, and `false` otherwise.

Example 1:

Input: `x = 121`
Output: `true`
Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: `x = -121`
Output: `false`
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: `x = 10`
Output: `false`
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

C++ Auto

```
1 class Solution {
2 public:
3     bool isPalindrome(int x) {
4         if (x < 0) return false;
5         int ans = 0;
6         int temp = x;
7         while (x != 0) {
8             int n = 0;
9             n = x % 10;
10            if ((ans > INT_MAX / 10 || (ans < INT_MIN / 10))) {
11                return 0;
12            }
13
14            ans = ans * 10 + n;
15            x = x / 10;
16        }
17        if (temp == ans) return true;
18        else return false;
19    }
20 };
```

Saved

Ln 1, Col 1

Testcase Test Result

Description

Editorial

Solutions

Submissions

485. Max Consecutive Ones

Solved

N/A Topics Companies Hint

Given a binary array `nums`, return the *maximum number of consecutive 1's in the array*.

Example 1:

Input: `nums = [1,1,0,1,1,1]`
Output: `3`
Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.

Example 2:

Input: `nums = [1,0,1,1,0,1]`
Output: `2`

C++ Auto

```
1 class Solution {
2 public:
3     int findMaxConsecutiveOnes(vector<int>& nums) {
4         int maxCount = 0;
5         int count = 0;
6         int n = nums.size();
7         for (int i = 0; i < n; i++) {
8             if (nums[i] == 1)
9                 count++;
10            else {
11                maxCount = max(maxCount, count);
12                count = 0;
13            }
14        }
15        maxCount = max(maxCount, count);
16        return maxCount;
17    }
18 };
```